SOFTWARE QUALITY & TESTING · PART 1 OF 2 · DETAILED

Software Testing

Every type, level and technique —

Dr. Syed Mohd Faisal Malik · CSC-504 Software Engineering · based on the SQT & Quality course material

PART 1 · ROADMAP

How this detailed session flows

1

1. Foundations

What testing is; error / defect / failure

2

2. Defects

Priority & severity, in depth

3

3. Methodologies

Black-box vs white-box

4

4. Types of testing

Functional, performance, usability, negative…

5

5. Levels

Unit → component → integration → system → UAT

6

6. Approaches & design

Integration approaches; EP, BVA; V&V; defects

SECTION 1

Foundations

What testing really is, why bugs happen, and the error–defect–failure chain

What is software testing?

What it means: An activity to ensure the correctness, completeness and quality of a system against its requirements. Put bluntly, it is a DESTRUCTIVE process — you execute the program deliberately trying to FIND errors and break it, so that defects surface in the lab and not in front of the customer.

Example: You don't test "login" only with the correct password. You try a wrong password, a blank field, a 200-character name, and odd symbols — actively trying to make it fail.

Analogy: *A car crash-tester does not gently drive the car around the block; they slam it into a wall to expose every weakness before a customer ever sits in it.*

Key point: *Testing hunts for FAILURE — it does not merely confirm success.*

Why bugs exist — and why we test

WHY SOFTWARE HAS BUGS

- Miscommunication (or none at all)
- Software complexity
- Programming errors
- Changing requirements
- Time pressure
- Egos
- Poorly documented code

WHY WE TEST

- Detect bugs before the customer does
- Build confidence the software works
- Check functional AND non-functional needs
- The product's quality & reliability depend on it
- Catch defects early — far cheaper than late

Bugs are inevitable and mostly human in origin — which is exactly why disciplined testing matters.

Error vs Defect (Bug) vs Failure

What it means: Three linked terms. An ERROR is a human mistake by the developer. That mistake becomes a DEFECT (bug) — the flaw sitting in the code. When the defective code runs and misbehaves, the visible wrong behaviour is a FAILURE.

Example: A developer types > instead of ≥ (error) → the wrong comparison sits in the code (defect) → at runtime the system wrongly rejects a valid boundary value such as age 18 (failure).

Analogy: *A cook misreads the recipe (error) → the dish is made with salt instead of sugar (defect) → the guest bites in and grimaces (failure).*

Key point: *Error → Defect → Failure = mistake, then flaw, then visible misbehaviour.*

CHECK YOUR UNDERSTANDING

Checkpoint 1 — Foundations

Q1 A tester keeps entering only correct data to “confirm the app works.” Why is this the wrong testing mindset?

Q2 A developer writes the wrong loop condition, and at runtime the report shows wrong totals. Which part is the error, which is the defect, and which is the failure?

Q3 Give one reason from today why software almost always contains bugs.

Discuss in pairs — then we’ll review the answers together.

SECTION 2

Defects — Priority & Severity

How urgently to fix vs how badly it hurts — and why they are independent

Priority — how URGENT

What it means: Priority answers how URGENTLY a defect must be fixed — the ORDER in which the team should tackle it, from P1 (fix first, most important) down to P5 (fix last, least important). It is largely a BUSINESS decision, weighing customer impact, visibility and deadlines.

Example: A wrong company phone number on the landing page is a tiny technical flaw — but it is P1, because customers see it instantly and it damages the brand, so it is fixed now.

Analogy: *The triage nurse in a busy emergency room deciding who is seen FIRST — not who is theoretically sickest, but who must be treated right now.*

Key point: *Priority = the fix-order / urgency — a business call.*

Severity — how BAD

What it means: Severity answers how BADLY the defect damages the system — the size of its technical impact — and is a TECHNICAL judgement, independent of how many users notice it. Levels, most-to-least serious: Blocker → Critical → Major → Normal → Minor → Suggestion.

Example: Blocker: halts all further testing/dev. Critical: a crash or data loss. Major: a key function broken. Normal: a small deviation. Minor: a tiny issue with an easy workaround. Suggestion: a proposal, not a fault.

Analogy: *Severity is how serious the injury is medically — a deep internal wound is high-severity even if unseen; a paper cut is low-severity even if it is on your face.*

Key point: *Severity = how much technical damage — a technical call.*

THE KEY INSIGHT

Priority × Severity — they are independent

Because one is a business call and the other a technical call, a defect can sit in ANY of the four corners — and this is a favourite interview question.

High severity + High priority

Payment fails for everyone → crashes AND is urgent. Fix immediately.

High severity + Low priority

App crashes on a rarely-used admin export → serious, but almost nobody hits it.

Low severity + High priority

A spelling mistake in the company logo on the home page → harmless, but everyone sees it.

Low severity + Low priority

A minor colour mismatch on an inner page → neither damaging nor urgent.

CHECK YOUR UNDERSTANDING

Checkpoint 2 — Priority & Severity

Q1 The company logo is misspelt on the home page. Is this high or low SEVERITY, and high or low PRIORITY? Why?

Q2 The app crashes — but only on a screen used once a year by one admin. Classify its severity and priority.

Q3 In one line: what is the fundamental difference between priority and severity?

Discuss in pairs — then we'll review the answers together.

SECTION 3

Testing Methodologies

Two fundamental ways to test: from the outside, or from the inside

Black-box (Behavioural) testing

What it means: Black-box testing treats the software as a sealed box: the tester knows only the INPUTS and the EXPECTED OUTPUTS, never the internal code. Test cases are built purely from the specification — “what should it do?”, not “how?”. It covers functional and non-functional testing and needs no programming knowledge.

Example: To test a login screen you try valid and invalid username/password combinations and check the results — without ever opening the source code.

Analogy: *Judging a car by test-driving it: you press the accelerator and see if it speeds up. You never open the bonnet to inspect the engine.*

Key point: *Black-box = judge behaviour from OUTSIDE, against the spec.*

White-box (Structural) testing

What it means: White-box testing (also clear-box, glass-box, transparent-box) looks INSIDE: the tester examines the internal code, logic and paths and designs tests to exercise them. It needs programming skill and knowledge of the code, and is usually done at the unit level — it verifies the code works the way it was written.

Example: Given an if (marks ≥ 40) branch, the tester writes cases that force BOTH the true and the false path to run, checking each line of the logic.

Analogy: *A mechanic inspecting the engine itself — checking each belt, valve and wire — not just driving the car around the block.*

Key point: *White-box = inspect the internal machinery; needs code knowledge.*

METHODOLOGY

Grey-box (Hybrid) testing

What it means: Grey-box testing sits between black-box and white-box: the tester has PARTIAL knowledge of the internals — things like the database structure, the algorithms, or the APIs — but still tests mainly from the outside, through the user interface. That inside knowledge is used only to design SMARTER, better-targeted tests, not to check the code line by line.

Example: A tester who knows the app stores dates as YYYY-MM-DD and ages as a whole number deliberately enters “31-02-2026” and an age of “−5” — inputs chosen precisely because they know how the data is stored behind the screen.

Analogy: *A food critic who has also read the recipe: they still judge the dish by eating it (from the outside), but knowing it contains saffron, they deliberately check whether the saffron flavour comes through.*

Key point: *Grey-box = outside testing guided by partial inside knowledge — common for web, database and integration testing.*

CHECK YOUR UNDERSTANDING

Checkpoint 3 — Methodologies

Q1 A tester has no access to the code and writes test cases only from the requirements document. Which methodology is this, and what is another name for it?

Q2 Which methodology would a developer use to make sure every branch of an `if–else` actually runs, and why does it need programming knowledge?

Q3 Which methodology is normally applied at the unit level?

Discuss in pairs — then we'll review the answers together.

SECTION 4

Types of Testing

Functional, performance, compatibility, usability, negative, ad-hoc, exhaustive

Functional testing

What it means: Functional testing checks WHETHER each feature works according to the stated requirement — the actions the system is supposed to perform. It is the heart of black-box testing and asks the simplest question: does the app actually do what it is meant to do?

Example: On a shopping site: does “Add to Cart” really add the item? does “Checkout” show the correct total? does the “forgot password” email actually arrive?

Analogy: *Testing a new microwave by actually cooking with it — does the timer start, does it heat the food, does opening the door stop it?*

Key point: *Functional = does each feature do its job?*

Performance testing

What it means: Performance testing measures HOW WELL the system performs its work — its speed and effectiveness: response time, throughput, and qualities such as reliability, scalability and interoperability. It is non-functional: the feature may work, but is it fast and steady enough?

Example: Confirming a product search returns results in under 2 seconds, and that the site still responds smoothly when 5,000 users shop at the same time.

Analogy: *Timing a chef during the dinner rush — not "can they cook?" but "can they serve 200 covers quickly without the kitchen falling apart?"*

Key point: *Performance = speed & effectiveness, especially under load.*

Compatibility testing

What it means: Compatibility testing checks that the software works correctly across different ENVIRONMENTS — web browsers, operating systems, hardware, devices, and even user languages. It is non-functional, and can be done manually or with automation tools.

Example: Confirming the site looks and behaves correctly on Chrome, Firefox, Safari and Edge; on Windows, macOS and Android; and on both a phone and a desktop screen.

Analogy: *Checking a plug fits every socket in the house — the appliance itself is fine, but does it actually work in each room's outlet?*

Key point: *Compatibility = works across browsers, OS, devices & users.*

Usability testing

What it means: Usability testing checks whether the interface is EASY to use and understand — the layout, wording and messages, cursor position and navigation, sort order of lists, and whether the project's screen standards (colours, field lengths, error highlighting) are followed.

Example: Is the cursor already in the first field when the page loads? Are error messages clear and shown in red? Is the menu sorted sensibly? Can a first-time user finish the task without help?

Analogy: *Handing a new gadget to your grandparent — if they can use it without the manual, it is usable; if they get stuck, it is not.*

Key point: *Usability = is the interface easy and clear to use?*

Negative testing

What it means: Negative testing deliberately feeds NON-RECOMMENDED, invalid values, trying to break the application. The software should REJECT them gracefully — with a clear error — rather than accept them or crash. It proves the app is safe on bad input, not just on good input.

Example: Typing alphabets into a numeric “salary” field; a future date in “date of birth”; numbers in a “name” field; a password shorter than the minimum length.

Analogy: *A bouncer checking IDs at a club — the whole point is to see the system correctly REFUSE what it should not allow in.*

Key point: *Negative = feed bad input; it must refuse, not break.*

Ad-hoc & Exhaustive testing

AD-HOC (Exploratory)

- Informal testing with NO formal test plan
- A skilled tester explores freely, following hunches
- Often finds defects that scripted tests miss
- Sometimes the only option when time is very short
- Analogy: wandering a new city with no map — you stumble on things a fixed tour would skip

EXHAUSTIVE

- Run one test case over ALL possible data
- Every combination of valid AND invalid inputs
- Combinations explode — so it is usually automated
- Gives the deepest coverage, at the highest cost
- Analogy: trying every key on a giant keyring — thorough, but only practical with a machine

Ad-hoc = unplanned but insightful; Exhaustive = total coverage, only practical through automation.

CHECK YOUR UNDERSTANDING

Checkpoint 4 — Types of testing

Q1 A tester enters letters into a field that should accept only numbers, expecting an error message. Which TYPE of testing is this?

Q2 The login feature works, but the team wants to know if the site survives 10,000 users at once. Functional or performance testing — and why?

Q3 Name the type that checks the app on Chrome, Safari and Edge, and the type that checks whether the screen layout is easy to understand.

Discuss in pairs — then we'll review the answers together.

SECTION 5

Levels of Testing

From the smallest unit up to the customer’s acceptance

Unit testing

What it means: The most “micro” level — testing a single function or code module ON ITS OWN, in isolation. It is done by the DEVELOPER (not testers) as the component is built, because it needs detailed knowledge of the internal design and code. It confirms each smallest piece is logically correct.

Example: Testing a calculateTax(amount) function alone with several sample amounts, long before it is ever wired into the billing screen.

Analogy: *A watchmaker checking that each tiny gear spins correctly BEFORE assembling the whole watch.*

Key point: *Unit = test the smallest piece alone — done by the developer.*

Component (Module) testing

What it means: Component (module) testing tests each individual software MODULE as a complete unit of functionality — one level up from a single function.

Example: An application has three modules: Add, Edit and Delete. Component testing checks each module fully and independently before any of them are combined.

Analogy: *Testing the whole engine on a workshop test-bench before it is ever dropped into the car.*

Key point: *Component = test each complete module on its own.*

Integration testing

What it means: Integration testing tests COMBINED modules working together, to check they cooperate correctly. The “parts” can be code modules, whole applications, or client and server pieces across a network — so it is especially important for distributed systems.

Example: After unit-testing “Login” and “Cart” separately, integration testing checks that logging in correctly carries the user’s cart across to the next screen.

Analogy: *After each musician has practised alone, the first rehearsal where the whole band plays TOGETHER — do they stay in time with one another?*

Key point: *Integration = do the combined parts work together?*

System testing

What it means: System testing tests the COMPLETE, integrated system end-to-end, evaluating it against the specified requirements — both functional behaviour and non-functional qualities such as reliability, security and compatibility.

Example: The ballpoint pen: cap, body, ink cartridge and tip are each unit-tested; assembled parts are integration-tested; and the finished, working pen is SYSTEM-tested — does it actually write?

Analogy: *The full dress rehearsal of a play — every actor, costume, light and cue running together as one complete show.*

Key point: *System = test the whole product end-to-end.*

System Integration Testing (SIT)

What it means: System Integration Testing verifies that the system correctly connects to any EXTERNAL or third-party systems it depends on, as defined in the requirements — not just that its own internal modules work.

Example: Checking that the app's checkout correctly talks to the external payment gateway and the third-party SMS provider, exchanging the right data both ways.

Analogy: *Testing that your new home-theatre actually communicates with the TV, the set-top box and the WiFi — not just that each device switches on by itself.*

Key point: *SIT = does our system talk correctly to external systems?*

User Acceptance Testing (UAT)

What it means: UAT is the FINAL testing, done by the END-USERS or the customer (sometimes over a limited trial period), to CONFIRM the system meets the mutually agreed requirements. It is the customer's sign-off before go-live.

Example: Before launch, the client's staff use the new billing system on real, day-to-day tasks for a week and confirm it does everything that was agreed.

Analogy: *The customer test-driving the finished custom car before accepting delivery and paying — "yes, this is exactly what I ordered."*

Key point: *UAT = the customer confirms it is what they agreed to.*

Alpha vs Beta testing

ALPHA TESTING

- Done in-house, BEFORE public release
- By an independent test team (or users)
- At the DEVELOPER'S own site / controlled setting
- Catches issues while the team can still react quickly
- Analogy: a chef tasting the new dish in their own kitchen before it reaches any diner

BETA TESTING

- Done by real END-USERS at THEIR own sites
- A beta version is released to the public
- Users explore it and report any defects they hit
- Reveals real-world conditions the lab never saw
- Analogy: handing out free samples in the market and collecting customers' honest reactions

Alpha = inside, before release; Beta = outside, by real users, in the real world.

CHECK YOUR UNDERSTANDING

Checkpoint 5 — Levels of testing

Q1 Put these in order from smallest to largest scope: System, Unit, Integration, Component.

Q2 Using the ballpoint-pen example: which level tests a single ink cartridge, and which tests the finished pen?

Q3 Who performs unit testing, and who performs UAT — and why the difference?

Discuss in pairs — then we'll review the answers together.

SECTION 6

Approaches & Techniques

Integration approaches, build & change tests, and how test cases are designed

Big Bang integration

What it means: All (or most) modules are coupled together AT ONCE to form the whole system, and then integration-tested in a single go. It saves set-up time, but if a fault appears, isolating WHICH module caused it among all of them is very hard.

Example: A team finishes all 12 modules, wires them together on the same day, and runs the whole app — then spends days hunting which connection broke it.

Analogy: *Assembling an entire flat-pack wardrobe without checking a single joint, then leaning on it — if it wobbles, which of the 40 screws is loose?*

Key point: *Big Bang = combine everything at once; fast to set up, hard to debug.*

Bottom-Up integration

What it means: Start by testing the LOWEST-level modules first, then use them to test the next level up, repeating until the top module is reached. It uses small “drivers” to call the lower modules. Its big advantage: bugs are found more easily.

Example: First test the low-level “database read/write” module, then the “order” module that uses it, then the “checkout” screen on top.

Analogy: *Building a house from the foundation upward — each lower floor is proven solid before the next one is added.*

Key point: *Bottom-up = lowest modules first, upward; bugs are easy to find.*

Top-Down integration

What it means: Start by testing the TOP (main) modules first, then work DOWN each branch step by step until the lowest modules. It uses “stubs” to stand in for lower modules that are not ready yet. Its big advantage: a missing branch or link is easy to spot early.

Example: Test the main menu and its high-level flow first (with dummy stubs below), then progressively replace stubs with the real sub-modules.

Analogy: *Sketching a painting’s whole outline first, then filling in details branch by branch — you notice immediately if an entire section is missing.*

Key point: *Top-down = top modules first, downward; missing links easy to spot.*

Sandwich integration

What it means: Sandwich (hybrid) integration combines top-down and bottom-up AT THE SAME TIME — the upper layers are tested downward while the lower layers are tested upward, meeting in the middle. It gains the benefits of both, at the cost of more complexity.

Example: One sub-team drives the top menus down with stubs, while another proves the low-level data modules upward with drivers — they join at the middle layer.

Analogy: *Digging a tunnel from BOTH ends of the mountain at once to meet in the middle — faster overall, but it needs careful coordination.*

Key point: *Sandwich = top-down and bottom-up together; best of both, more complex.*

CHECK YOUR UNDERSTANDING

Checkpoint 6 — Integration approaches

Q1 A team joins all modules together on one day and tests the whole thing at once. Name the approach and its main drawback.

Q2 Which approach tests the lowest-level modules first, and what is its main advantage?

Q3 Top-down testing uses “stubs” and bottom-up uses “drivers.” In one line each, what is a stub and what is a driver?

Discuss in pairs — then we’ll review the answers together.

BVT, Smoke & Sanity testing

BVT	Build Verification Test — a short set of tests run on EACH new build to confirm it is stable enough to hand to the test team. A build that fails BVT is rejected, and testing continues on the previous build.
SMOKE	Wide & shallow. Checks the most crucial functions of the WHOLE app work, without going deep. Scripted/automated, run on every build. (Origin: switch on new hardware — if it does not catch fire, it passes.)
SANITY	Narrow & deep. After a small change or fix, checks that the ONE affected area now works and nothing obvious broke. Usually unscripted.

Smoke = broad "is the build even testable?" · Sanity = narrow "did this specific fix work?"

Re-testing vs Regression testing

RE-TESTING

- Re-run ONLY the test(s) that had failed
- Confirms the specific reported defect is now fixed
- Narrow and focused on that one thing
- Uses the SAME data that exposed the bug

REGRESSION

- Re-run a broader set of tests around the change
- Confirms the fix did not BREAK anything else
- Purpose: ensure NO new defects were introduced
- Run frequently to hold a quality baseline

Analogy: after a plumber fixes the kitchen tap, RE-TEST that tap — then REGRESSION-check every other tap and toilet still works.

Load, Stress & Volume testing

LOAD

System under EXPECTED, realistic load — many users doing normal work at the same time. e.g. thousands of shoppers browsing at once, or many jobs queued to a printer.

STRESS

System pushed BEYOND normal limits, until it breaks — too little memory or disk, a crash situation. It finds the breaking point and checks the system fails gracefully.

VOLUME

System with very large AMOUNTS of data — the database filled to maximum, then searched and sorted. It finds where big data makes performance or usability degrade.

Load = many USERS · Stress = past the LIMIT · Volume = huge DATA.

Equivalence Partitioning (EP)

What it means: **G**roup the possible inputs into CLASSES that the program treats the same way, then test just ONE value from each class — the other values in that class would find nothing new. This cuts a huge (even infinite) set of inputs down to a handful of meaningful tests.

Example — awarding a grade from a mark (0–100):

0–39 → D	**40–59** → C	**60–70** → B	**71–100** → A
< 0 → Invalid	**> 100** → Invalid	**non-numeric** → Invalid	

Analogy: *you don't taste every spoonful from the same pot of dal — one spoon represents the whole pot. Pick ONE mark per class (e.g. 20, 50, 65, 85, –12, 150, "abc"): 7 tests, full coverage.*

Boundary Value Analysis (BVA)

What it means: bugs cluster at the EDGES of a range — off-by-one errors like using > instead of ≥, or starting a range at 0. So instead of testing the middle, you test right AT each boundary: a valid value and an invalid value on each side.

Example — a "month" input, valid range 1–12:



Analogy: *accidents happen at the EDGE of the cliff, not the middle of the field — so test the edges. Rule: for a boundary n, test n−1, n, n+1. Always pair BVA with EP: EP finds the classes, BVA tests their boundaries.*

CHECK YOUR UNDERSTANDING

Checkpoint 7 — Design techniques & change tests

Q1 A field accepts ages 18–60. Using EP and BVA together, list the boundary values you would test at the lower edge.

Q2 After developers fix a login bug, which testing re-checks ONLY that login, and which re-checks that the rest of the app still works?

Q3 A build is combined and you quickly check just the crucial functions work before deep testing. Is that smoke or sanity testing?

Discuss in pairs — then we'll review the answers together.

Reviews (static testing)

What it means: A review examines a work product — SRS, test cases, code, plans — by a person or group, to catch defects EARLY, without running any code (this is STATIC testing). Informal types: peer review, walkthrough, SME review. Formal types: inspection (by independent auditors) and group reviews. Running the code with test cases is, by contrast, DYNAMIC testing.

Example: Before a single line runs, three engineers read the SRS together and spot a contradictory requirement — fixed on paper in minutes instead of after weeks of coding.

Analogy: *Proof-reading an exam paper before it is printed — far cheaper than reprinting thousands of copies after a mistake is found.*

Key point: *Reviews are static (no code run) and catch defects at their cheapest — "better a peer finds it than the customer."*

Verification vs Validation

VERIFICATION

"Have we built the software RIGHT?"

- Does it match the specification?
- STATIC — no code run (reviews, checks)
- Done at each phase vs that phase's inputs
- "Are we building it correctly?"

VALIDATION

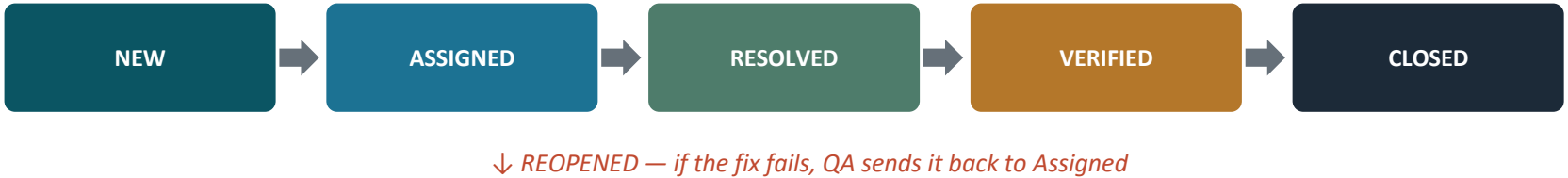
"Have we built the RIGHT software?"

- Is this what the customer actually wants?
- DYNAMIC — executing the software
- Done during / at the end of development
- "Are we building the correct thing?"

Analogy: *verification = did we bake the cake exactly per the recipe; validation = is this the cake the customer actually wanted.*

Debugging & the defect life cycle

Debugging is the developer's process of LOCATING and FIXING (or bypassing) the cause of a failure — distinct from testing, which only FINDS the failure. Each defect then travels through a status life cycle:



The journey: QA logs a NEW bug → the lead validates it and ASSIGNS it to a developer → the developer RESOLVES (fixes) it → QA VERIFIES the fix → it is CLOSED. If the fix did not work, QA REOPENS it. Afterwards, Root Cause Analysis asks WHY the defect happened — so the same type of defect can be prevented in future.

CHECK YOUR UNDERSTANDING

Checkpoint 8 — Reviews, V&V & defects

Q1 Three engineers read the SRS together and find a contradictory requirement before any coding. Is this static or dynamic testing, and what is it called?

Q2 Match each to verification or validation: (a) “Does it match the specification?” (b) “Is this what the customer actually wanted?”

Q3 A tester verifies a fixed bug and it still fails. What status does the bug move to, and who moves it there?

Discuss in pairs — then we’ll review the answers together.

SECTION 7

Testing Artefacts & Life Cycles

How tests are executed and documented — cases, plans, and the two life cycles

Manual vs Automation testing

MANUAL TESTING

- A human executes the test cases by hand
- Best for usability, ad-hoc / exploratory testing
- Good for small, one-off or first-time tests
- Human judgement notices “odd” things
- Slower & tiring for large, repetitive test sets

AUTOMATION TESTING

- A tool runs the test cases via scripts
- Best for repetitive, large & regression tests
- Ideal for exhaustive tests — one case over MANY data values
- Fast, repeatable, runs overnight
- Needs scripting effort, tools & skilled people

Rule of thumb: automate the repetitive, high-volume, oft-repeated checks; keep humans for judgement and exploration.

Test Case vs Test Scenario

TEST CASE

- A description of WHAT to test
- WHAT data to use, WHAT action to perform
- The EXPECTED result, to compare with the actual
- Formal steps + instructions
- Types: functional (smoke/component/integration/system), usability, negative, performance

TEST SCENARIO

- A higher-level SITUATION to test
- An end-to-end business flow
- “What situations do we want to test?”
- e.g. search a bank customer by:
- name · account number · customer ID · mobile number

A scenario is the SITUATION; a test case is the concrete steps + data + expected result that cover it.

The Test Plan

What it means: The Test Plan is one of the most significant testing documents — created by the Test Manager, Test Lead or senior testers after gathering input from all stakeholders. It sets out HOW the whole testing effort will be run, so the team works to one agreed plan.

Example: A typical plan captures: Scope · test Environment · Schedule · Risks · Resources · Execution approach · Reporting · Automation · and the Completion (exit) criteria.

Analogy: *A film shoot's production plan — who does what, when, where, with which equipment, and when the shoot counts as "done" — all agreed before a single scene is filmed.*

Key point: *The test plan answers: who tests WHAT, HOW, WHEN, and when testing is "done."*

Software Testing Life Cycle (STLC)

What it means: the ordered sequence of phases a TESTING effort passes through. Crucially, testing begins in the requirements phase — not after coding is finished.

1. Requirements analysis
2. Test planning
3. Test development (cases & scripts)
4. Test execution
5. Test reporting
6. Defect analysis
7. Re-testing
8. Regression testing
9. Test closure (archive & lessons)

Analogy: *a professional photo shoot — understand the brief → plan → prepare the set → shoot → review the shots → reshoot the bad ones → final check → wrap & archive.*

Defect (Bug) Life Cycle

What it means: every defect travels through a set of STATUSES, from the moment QA finds it to the moment it is closed.



↓ *REOPENED — if the fix fails, QA sends it back to ASSIGNED*

NEW — QA logs a newly found bug

ASSIGNED — Lead validates it & assigns to a developer

RESOLVED — Developer fixes it (or marks Later / Invalid)

VERIFIED — QA re-checks and confirms the fix works

CLOSED — Fixed & verified — the bug is closed

REOPENED — Fix failed — QA reopens it (back to Assigned)

Analogy: *a hospital patient — admitted (New) → assigned a doctor (Assigned) → treated (Resolved) → re-checked (Verified) → discharged (Closed); if symptoms return, re-admitted (Reopened).*

CHECK YOUR UNDERSTANDING

Checkpoint 9 — Artefacts & life cycles

Q1 Put these STLC phases in the correct order: Test execution · Requirements analysis · Test closure · Test planning.

Q2 A tester logs a bug, the lead assigns it, the developer fixes it, and QA confirms the fix. List the bug statuses it passes through.

Q3 In one line each: what is a test case, and what is a test scenario?

Discuss in pairs — then we'll review the answers together.

Part 1 — the whole picture

Testing a destructive process — execute the software to FIND errors.

Defects severity = how BAD (technical); priority = how URGENT (business).

Two methods black-box (behaviour, from spec) vs white-box (internal code).

Levels unit → component → integration → system → UAT.

Design equivalence partitioning + boundary value analysis.

V & V verification = built it right; validation = built the right thing.